MANIPAL UNIVERSITY JAIPUR

Computer Science and Engineering

DATA STRUCTURES AND ALGORITHMS

PROJECT REPORT

Smart Campus Navigation System

A graph-based web application for outdoor and multi-floor indoor navigation

---

Submitted by

Aaditya Shah

BTech Computer Science and Engineering

Registration number: _____

Course and section: _____

Submitted to: _____

Submission date: 19 October 2026

Scope. A student DSA project. MUJ outdoor data is derived from OpenStreetMap and is incomplete. The indoor building is an explicitly fictional educational demonstration, not an actual MUJ floorplan. No official endorsement or field-survey certification is implied.

Implementation reference: `feat/muj-indoor-demo`, commit `136667f`. Demonstration runtime: Docker Desktop.

Abstract

Smart Campus Navigation System converts the campus navigation problem into a graph-search problem. Junctions, entrances, rooms and floor stops become vertices; permitted movements become directed edges. The route engine builds an adjacency list, filters ineligible connections and applies Dijkstra's algorithm using a binary min-heap. A predecessor map reconstructs the selected route for map overlays and textual instructions.

The application combines a React and TypeScript interface, a FastAPI backend and PostgreSQL with PostGIS. It supports directory search, route policies, administrative drafts, imports, publication, QR starts, user-triggered location and operational restrictions. A separate two-floor educational graph demonstrates indoor routing without inventing physical distances. This report emphasizes the algorithms first, then explains the system that makes their results usable and reproducible.

Keywords: directed graph; adjacency list; Dijkstra; priority queue; shortest path; indoor navigation.

Principal contributions

The project connects a pure route-search engine to a working directory, outdoor map and floor-aware educational viewer. It demonstrates multiple-source and multiple-target routing, policy-dependent edge costs, explicit no-route handling and recovery from stale application state. Administrative drafts and immutable releases separate data editing from the public graph.

Boundaries of the submission

The outdoor preparation is based on OpenStreetMap. The indoor example is an original, unscaled educational graph. It demonstrates the algorithm and user interaction without claiming surveyed MUJ rooms, complete campus connectivity, real-time tracking or verified physical accessibility.

Reading the evidence

Implementation statements are grounded in the repository version cited on the cover. Test counts describe recorded cloud runs, not a fresh test run performed while writing this report. Earlier Windows checks were owner-reported. Windows acceptance of the final integrated indoor tabs remains pending in the available evidence. Illustrations labelled conceptual explain algorithms and are not geographic campus maps.

Contents

1 Project problem and DSA objectives 4
2 Directed graphs and movement semantics 5
3 Adjacency lists and graph construction 6
4 Dijkstra algorithm and correctness 7
5 Worked shortest path example 8
6 Min-heap and priority queue 9
7 Hashing, sets, lists and reconstruction 10
8 BFS, DFS and complexity analysis 11
9 System architecture and request flow 12
10 Database design and release lifecycle 13
11 MUJ outdoor data and route policies 14
12 Indoor demonstration model 15
13 Indoor computation and floor transitions 16
14 Public interface and state management 17
15 Administrative graph and import workflows 18
16 QR, location, closures and eligibility 19
17 Verification and observed results 20
18 Operation, limitations and future work 21
19 Conclusion and references 22

Reading order. Sections 1–8 explain the data structures and algorithms. Sections 9–16 connect them to the application. Sections 17–19 present verification, limitations and conclusions.

1 Project problem and DSA objectives

A campus directory answers where a place is listed, but navigation must answer how a person can reach it. Straight-line distance is insufficient because walls, disconnected paths, one-way movement, stairs, missing entrances and closures constrain movement. SCNS therefore separates visible geometry from valid graph connectivity. A place can appear on a map while having no eligible route.

Formal problem

Given a directed graph G = (V, E), a set of valid source anchors, a set of destination anchors and a routing policy, find a permitted path of minimum total non-negative cost. Return the ordered stored edges and instructions, or an explicit no-route result. The chosen cost can represent stored distance, supported traversal time or one unit per eligible traversal; these objectives must remain distinct.

DSA concept	Concrete role in SCNS
Directed weighted graph	Represents movement and its permitted direction
Adjacency list	Groups outgoing eligible arcs by source vertex
Binary min-heap	Chooses the next lowest-cost frontier entry
Dictionary and set	Store best ranks, predecessors and identity membership
Lists	Hold arcs, route nodes, route edges and display segments
Graph reachability	Explains disconnected rooms and missing routes

Optimization objective

For eligible paths P between source anchors S and target anchors T,

$$P^* = \arg\min_{P:S \rightsquigarrow T} \sum_{e \in P} w(e).$$

Here $w(e)$ is stored distance, supported time, or 1 per traversal. SCNS accepts only finite, strictly positive movement costs. A zero-edge route can still have total cost zero.

Learning objectives

The project demonstrates modeling, algorithm choice, correctness reasoning and the connection between computational results and user behavior. It also makes the consequences of incomplete input visible: an algorithm cannot repair an unknown door or prove accessibility from a drawing. The application is useful as a DSA case study because algorithm behavior can be inspected through real route selections and a deliberately disconnected indoor room.

2 Directed graphs and movement semantics

A vertex identifies a topological position, not simply a point on a picture. Outdoor vertices represent stored junctions or gates. Indoor vertices belong to a floor and coordinate frame. An anchor connects a searchable destination to a graph vertex. Separate connector records describe movement between floors or between indoor and outdoor spaces when that relationship is explicitly mapped.

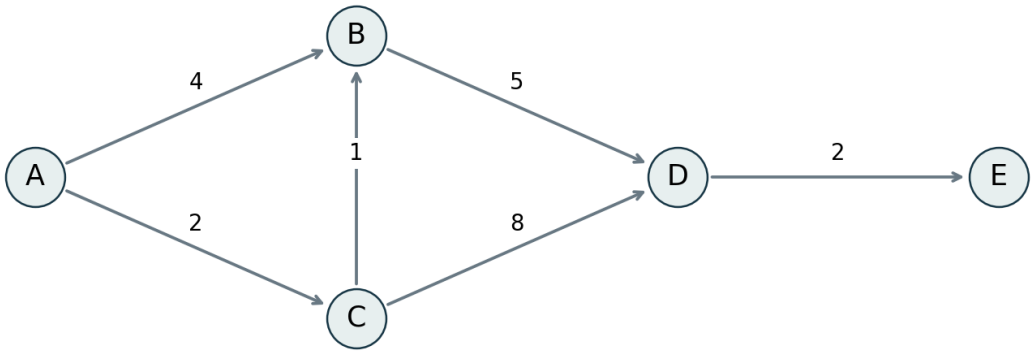


Figure 1: Conceptual directed weighted graph used in the worked example. Weights are illustrative cost units.

In Figure 1, A can reach B or C. The arrow C $\rightarrow$ B is a separate movement from B $\rightarrow$ C; a return direction does not exist unless authored. Two geometrically crossing lines do not automatically share a vertex. Conversely, two records with similar coordinates must not be merged solely because they look close.

Positive costs and eligibility

The Arc data class rejects non-finite and non-positive costs. The graph builder excludes closed or non-public edges. A step-free query excludes stairs and any edge whose accessibility is not explicitly verified as step-free. The shortest route is therefore the shortest route within the filtered graph, rather than the shortest route through every drawn line.

This model handles one-way paths and disconnected components naturally. If either endpoint has no eligible anchor, the request cannot be routed. If anchors exist but lie in separate reachable regions, the frontier eventually empties and the engine returns no path. The UI should explain this result without drawing an invented connection.

3 Adjacency lists and graph construction

SCNS uses a dictionary of lists to represent adjacency. Each key is a source node identifier, and its value is the list of Arc records leaving that node. This representation is appropriate for a campus graph, where most locations connect to only a few nearby junctions rather than to every other vertex.

```
adjacency = defaultdict(list)
for arc in arcs:
    reject duplicate edge identifiers
    skip closed or non-public arcs
    skip arcs excluded by the accessibility profile
    adjacency[arc.source].append(arc)
for outgoing in adjacency.values():
    outgoing.sort(key=lambda arc: arc.id)
```

Vertex	Outgoing neighbor and cost
A	B with 4; C with 2
B	D with 5
C	B with 1; D with 8
D	E with 2
E	No outgoing connection

Why this representation fits

The adjacency list stores the graph in O(V + E) space in the standard model. Expanding a node inspects its outgoing edges rather than scanning all possible destinations. A dense adjacency matrix would require $O(V^2)$ space, including many empty relationships. Lists also retain edge identifiers and metadata needed for route reconstruction and instructions.

The implementation sorts outgoing edges by identifier to make behavior deterministic. It also uses a set to reject duplicate edge IDs before routing. These preprocessing details matter when equal-cost choices exist or when imported data contains accidental duplicates. Geometry remains associated with the edge record; Dijkstra chooses identifiers, and the service later converts those chosen records into route segments.

4 Dijkstra algorithm and correctness

Dijkstra's algorithm repeatedly expands the frontier entry with the smallest known accumulated cost. The engine initializes all eligible source nodes with cost zero, making it a multi-source search. It stores the best known rank for each reached node and remembers the incoming edge that produced an improvement. Destination candidates can also be multiple anchors of the same place.

```
best = zero-cost rank for every eligible source
heap = source entries ordered by source ID
heapify(heap)  # rearranges the list in place
previous = {}; terminal = None
while heap is not empty:
    rank, serial, u = heappop(heap)
    if terminal exists and rank.cost > terminal.cost: break
    if rank != best[u]: continue  # stale candidate
    if u is a target: update terminal by cost/priorities; continue
    for arc in adjacency[u]:
        candidate = extend(rank, arc.cost, arc.priority)
        if candidate < best.get(arc.target, infinity):
            best[arc.target] = candidate
            previous[arc.target] = arc
            heappush(heap, (candidate, next(serial), arc.target))
return reconstructed terminal path, or None
```

Why greedy expansion is valid

All accepted arc costs are positive. Therefore any extension of a path cannot reduce its accumulated cost. When a current minimum-cost frontier entry is removed, an undiscovered cheaper route to that node cannot pass through a more expensive frontier prefix. This is the central argument behind Dijkstra's primary-cost correctness [10]. Negative-cost movement would violate that argument and is rejected by the graph model.

Implementation details beyond textbook pseudocode

The real engine compares tuples containing primary cost and secondary priorities, not just a scalar distance. Target priority and source priority help choose among equal-cost anchors. A serial counter prevents heap comparisons from depending on arbitrary payload ordering. After finding a target, the engine still considers equal-primary-cost candidates and stops only when the next relevant cost is greater than the terminal cost.

The pseudocode summarizes the algorithm. The authoritative implementation is backend/app/routing/dijkstra.py [1]. In the mixed service the Arc field named distance_m carries the selected optimization cost; the service separately tracks actual physical measurements and declares the correct cost units [2].

5 Worked shortest path example

Consider the conceptual graph in Figure 1. The task is to reach E from A. The example costs are deliberately small so that relaxation can be followed by hand. They are not measurements of MUJ paths. The first discovered A-to-B cost is 4, but visiting C first improves it to 3. This improvement changes the final route to E.

Expanded node	Relaxation or decision	Best costs after expansion
A at 0	B becomes 4; C becomes 2	A 0, B 4, C 2
C at 2	B improves to 3; D becomes 10	B 3, C 2, D 10
B at 3	D improves to 8	D 8
Old B at 4	Stale heap entry is skipped	No change
D at 8	E becomes 10	E 10
E at 10	Destination candidate found	Final cost 10

Predecessor chain

The predecessor map stores E $\leftarrow$ D, D $\leftarrow$ B, B $\leftarrow$ C and C $\leftarrow$ A. Following this chain backward gives E, D, B, C, A. Reversing it produces A $\rightarrow$ C $\rightarrow$ B $\rightarrow$ D $\rightarrow$ E. The edge cost sum is 2 + 1 + 5 + 2 = 10. The engine also returns the exact ordered edge IDs so instructions and map geometry use the same path.

Why the older B entry remains

The implementation pushes an improved entry instead of locating and decreasing a key already inside the heap. Both B at 4 and B at 3 may temporarily exist. On removal, the comparison with best[B] identifies the old rank as stale. This lazy-update approach keeps Python heap operations simple while preserving primary shortest-path correctness.

If the D $\rightarrow$ E connection is removed, E becomes unreachable from A in this example. The correct behavior is no path, not a route that crosses empty space. If source and destination share an eligible node, a zero-edge result is possible even though individual movement arcs require strictly positive costs.

6 Min-heap and priority queue

Python's heapq module supplies the binary min-heap used by the router [9]. A heap maintains a partial ordering: each parent has priority no greater than its children. It does not keep the whole collection fully sorted. The smallest entry is available at the root, which is exactly the operation Dijkstra requires at every expansion.

Operation	Purpose	Typical cost
heapify	Build initial multi-source frontier	O(S)
heappush	Add an improved candidate	O(log H)
heappop	Extract smallest candidate	O(log H)
Compare with best	Reject stale popped rank	Expected O(1) dictionary lookup

Here S is the number of source nodes and H is the current heap size. Heap construction is linear after the initial entries are available; the implementation separately sorts the sources in $O(S \log S)$ time. The conceptual entries after expanding A are C at 2 and B at 4. After expanding C, the heap can contain B at 3, B at 4 and D at 10. Only the root is guaranteed to be the minimum; the underlying list is not itself a sorted route.

Ranking and deterministic behavior

Queue entries include a rank tuple, a monotonically increasing serial value and a node ID. Lexicographic tuple comparison prioritizes the primary route cost first. Secondary source and path priorities resolve ties without overriding a cheaper route. The serial value supplies a stable distinction between otherwise equal queue entries.

Array representation of a binary heap

For zero-based index i, the children occur at $2i+1$ and $2i+2$. The parent, when $i > 0$, occurs at $\lfloor (i-1)/2 \rfloor$. The invariant is that each parent rank is no greater than its existing child ranks. This is a tree ordering stored in a Python list, not a separately implemented tree class.

Why a FIFO queue is different

A normal queue removes the earliest inserted entry, regardless of accumulated cost. That is useful for BFS on equal-weight graphs but does not solve arbitrary weighted shortest paths. Sorting the complete frontier before every extraction would perform unnecessary work. A binary heap provides efficient incremental insertion and minimum extraction without maintaining a full sorted order.

7 Hashing, sets, lists and reconstruction

Several simple structures work together around the graph algorithm. Python dictionaries are used as hash-based mappings: adjacency maps a node to outgoing arcs, best maps a node to its current rank, and previous maps a reached node to the incoming Arc. Sets support unique IDs and membership of sources, targets and closed edges. Lists preserve sequence for input arcs and final output.

Structure	Stored information	Reason for selection
best dictionary	Node ID → current best rank	Fast improvement and stale checks
previous dictionary	Node ID → incoming edge	Reconstruct one chosen path
seen set	Encountered edge identifiers	Reject duplicate edge IDs
sources and targets sets	Eligible endpoint node IDs	Fast endpoint membership
Path lists	Ordered nodes and edge IDs	Stable map and instruction order

Reverse then restore forward order

Reconstruction begins at the selected terminal node. At each step, the incoming edge is appended to an edge list and its source node is appended to a node list. The chain ends at a source because sources do not need a predecessor. Reversing both lists restores start-to-destination order. If the returned path contains L edges, reconstruction takes O(L) time and O(L) output space.

The reconstruction is iterative and uses list appends followed by reversal. It does not require recursion or a custom stack. Python lists are dynamic arrays; the submission does not claim a custom linked-list implementation. The binary heap provides a tree-based ordering, while the navigable campus itself remains a graph.

Complexity and identifiers

Dictionary and set operations are expected O(1), not an unconditional worst-case constant. Hash collisions and implementation behavior can affect the worst case. Stable UUID-based object identities allow records to be matched across APIs, graph references and release artifacts; the UUID text is an identifier, not a geographic coordinate or readable building name.

8 BFS, DFS and complexity analysis

The indoor demo assigns every authored connection a cost of one traversal. On a graph with equal edge weights, BFS would also find a path with the fewest traversals in O(V + E) time. The demo nevertheless reuses SCNS's existing Dijkstra engine, avoiding a second route implementation and making the change in cost policy explicit.

Method	Suitable objective	Complexity in standard model
BFS	Fewest edges on an unweighted graph	O(V + E)
DFS	Reachability and component exploration	O(V + E)
Heap Dijkstra	Minimum sum of non-negative weights	O((V + E) log V), common sparse simple graph bound
Adjacency matrix	Dense graph representation	$O(V^2)$ storage

Bound for the actual implementation

Let V and E be vertex and directed-arc counts, and let $S \leq V$ be the number of source nodes. Lazy updates may retain $O(E+S)$ heap entries. A conservative search bound is $O((V+E)\log(V+E))$, assuming constant-size rank comparisons and expected constant-time hashing. Preprocessing adds $O(S\log S)$ for source ordering and

$$\sum_{u \in V} O\left(d^+(u)\log\max\{2, d^+(u)\}\right)$$

for outgoing-arc ordering, bounded by $O(E\log E)$. Working space is $O(V+E)$. The customary $O((V+E)\log V)$ description is appropriate under the usual simple-graph assumptions; these are algorithmic bounds, not measured response times.

What changes the chosen route

Weighted routing minimizes stored metres, or supported seconds for an eligible fastest query. Directions-only routing minimizes traversals. These are different objectives: a nine-edge route can be physically longer than a thirteen-edge route. Also, adding intermediate graph nodes can change the traversal objective even if physical geometry is unchanged. Fewest traversals must not be described as fewest turns or shortest walking distance.

DFS explores reachability rather than minimum weighted cost. SCNS does not switch to BFS or DFS for public routes. Dijkstra optimizes the eligible stored graph; it cannot establish that the map fully represents the campus.

9 System architecture and request flow

The application separates presentation, business services, spatial persistence and background processing. React renders the campus directory, planners, maps and administrative screens. FastAPI validates requests and exposes typed API contracts. PostgreSQL stores application records; PostGIS supplies spatial types and operations. A bounded worker processes staged uploads and floorplan jobs outside the ordinary request path.

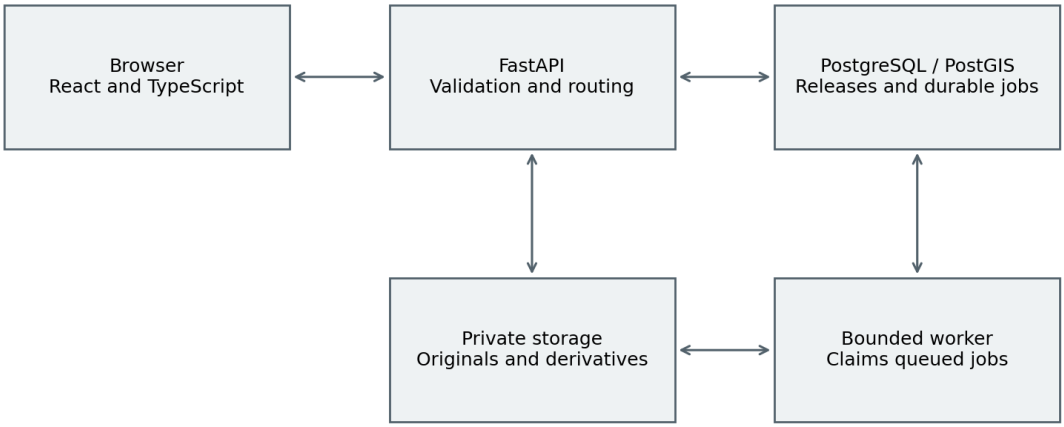


Figure 2: Runtime architecture: database-backed jobs and shared private storage. The educational demo reads a separate authored graph.

A normal route request

The browser obtains the active campus release and available places. The user chooses endpoints, policy and profile. The API resolves eligible anchors, applies release and restriction rules, assembles arcs and calls the pure routing engine. The result is converted into ordered map segments, floor metadata, instructions and measured totals where known. Frontend state checks prevent stale results from being treated as current after an operational change.

A deliberately smaller educational path

The endpoint `/api/v1/educational/indoor-school` returns the graph version and original floor schematics. Route requests reuse shortest_path and return floor-aware segments with null distance and duration. This read-only service uses no database session or publication workflow, keeping the demonstration separate from MUJ's release [3, 4].

The implemented separation allows algorithms to be unit-tested without a browser, APIs to be checked with isolated databases, and the browser to be tested against real containerized services. It also prevents visual changes from requiring graph or schema changes.

10 Database design and release lifecycle

Persistent data is organized around a campus and a dataset revision. Places describe discoverable destinations; buildings provide catalog context; nodes and edges describe topology; anchors connect destinations to movement nodes. Indoor floors, rooms, local coordinate frames and connector records extend the model without pretending that illustration pixels are latitude and longitude.

Record family	Stored responsibility
Campus and revision	Scope, metadata and editable dataset identity
Place building anchor	Directory entries and graph entry points
Node edge geometry	Directed topology and stored path shape
Floor room connector	Indoor catalog, local geometry and explicit transitions
Release artifact active release	Immutable published content and selected public version
User session role audit	Authentication, campus authorization and traceability
Import job private source	Durable staging and processing provenance

Drafts and public data

Editors work on a draft rather than altering the public release directly. Version checks protect against overwriting another operator's newer edit. Validation and review occur before publication. Publication produces immutable release content and updates which release is active. A rollback selects earlier published content; it is not a database reset or a destructive rewrite of history.

Why this matters to routing

A route should be computed from a coherent dataset rather than an accidental mixture of old nodes and new edges. Release identity and checksums help bind public responses to a particular projection. Operational restrictions are a separate concern and can exclude paths without requiring an editor to publish an entirely new campus map.

The educational indoor graph is application content and has no campus revision or release row. Its explicit version string detects stale client requests. The original production database is retained when code is updated; a Git push alone does not back up that database or its private uploaded files.

11 MUJ outdoor data and route policies

The MUJ outdoor preparation is derived from OpenStreetMap source records, not from inventing paths between visually nearby buildings. The reviewed preparation contains 51 buildings, 55 destinations, 295 graph nodes, 700 directed edges and 22 anchors. Its 29 weak components and 33 unanchored destinations explain many unavailable journeys. Weak connectivity ignores edge direction: membership in one component is necessary, but not sufficient, for a directed route. These are recorded v2 preparation figures, not a live database count [7].

Recorded query	Metric result	Interpretation
University Entrance → mapped entrance 10828149259	513.10 m over 13 edges	Stored-distance objective
Same endpoints in Directions only	799.32 m over 9 edges	Traversal objective; displayed metres are not optimized
Hostel Entrance → GHS Entrance	23.62 m over 3 edges	Supported mapped connection
Hostel Entrance → Faculty Block Entrance	163.28 m over 6 edges	Supported mapped connection
Hostel Entrance → Central Library	No eligible route	Missing source-supported entrance anchor

Interpreting incomplete evidence

An outline on a basemap is not automatically a walkable edge. Shared source nodes, valid boundary coverage, access rules and level separation must support the connection. In particular, an underground tunnel must not be joined to a road merely because their lines cross in plan view. Owner-supplied knowledge can guide future mapping, but should retain its source and uncertainty.

Unknown names are retained as mapped entrances rather than replaced with guessed building identities. The E22 entrance has a source-supported TMA Pai Auditorium name [8]. The tested E05 destination remains unidentified. OpenStreetMap attribution and applicable source conditions must remain visible when using its derived data.

These route values are recorded software results. They are not field-survey evidence, an accessibility certification or a guarantee that a gate is currently open. Operational restrictions can change the result on the presentation machine.

12 Indoor demonstration model

The MUJ explorer includes Outdoor and Indoor demo tabs in the same map section. The indoor mode displays an original Demo Learning Building with ground and first floors. It is explicitly labelled as a demonstration layout rather than an actual MUJ floorplan. The reference PDF informed only general organization; its architectural drawings are not shipped as the demo assets.

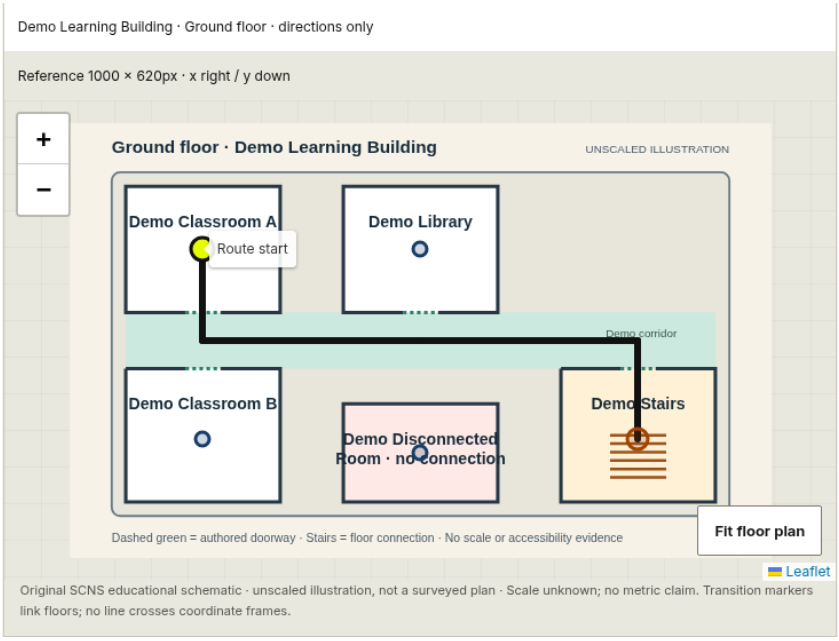


Figure 3: Repository test-fixture render of the integrated ground-floor demo. This is automated evidence, not a screenshot of the owner's Windows database.

The authored graph contains 15 nodes and 26 directed connections: seven rooms and eight corridor or stair nodes. Each of its 13 physical-style connections is represented in both directions. Six rooms are connected through the corridor and stair structure; the seventh is intentionally isolated. This creates an easy-to-explain reachability test without altering real MUJ data.

Coordinates are unscaled illustration pixels. The floors use separate coordinate frames, so the same pixel position on two floors does not identify the same vertex. Generic room names and explicit stairs make the model readable without implying surveyed indoor measurements or a verified accessible route.

13 Indoor computation and floor transitions

A request supplies a source room, destination room and expected demo version. The service rejects foreign room IDs, unsupported fields and stale versions. It converts each authored connection into an Arc with cost 1.0, calls the existing Dijkstra engine and maps the selected edges into floor-aware route segments.

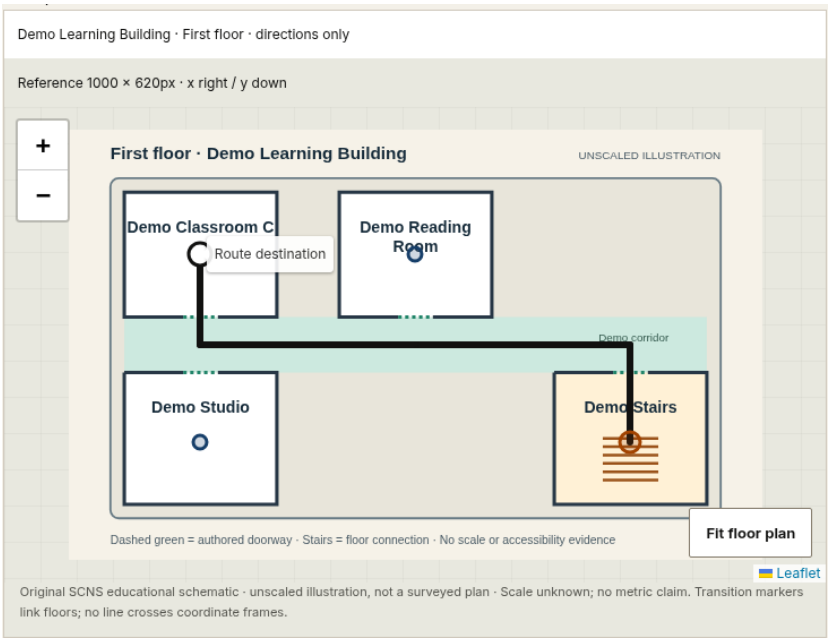


Figure 4: Repository test-fixture render of a computed cross-floor route. Floor switching reveals each local route section.

Demo query	Expected result
Classroom A → Library	3 authored traversals
Classroom A → Classroom C	9 traversals with stair ascent
Classroom C → Classroom A	9 traversals with stair descent
Classroom A → Disconnected Room	No route and no highlighted path
Classroom A → Classroom A	0 traversals; physical totals remain unknown

The stair edge links two floor stops semantically; it is not drawn as a diagonal line between unrelated floor images. Text identifies the transition and provides a Show First floor action. Distance and ETA remain null even when a route is found. This cleanly demonstrates that a topological path can be known while its physical length is not.

14 Public interface and state management

The public interface combines an editorial campus introduction with a full directory and explorer. MUJ-specific photo and emblem assets depend on campus metadata. Demo Campus keeps its synthetic disclosure and does not inherit MUJ branding. Search, filtering, pagination and Start here or Route here actions remain available alongside the map.

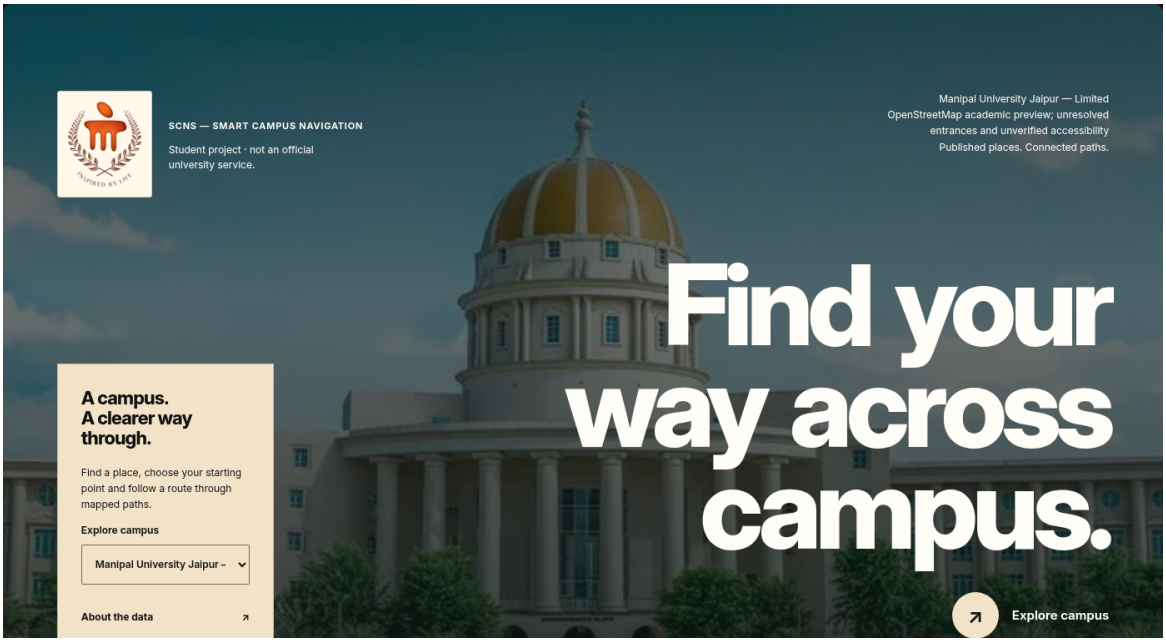


Figure 5: Upper portion of a repository MUJ branding test render. Campus metadata was simulated over isolated fixtures.

Separate mode state

Outdoor and Indoor demo retain their own endpoint selections, results, errors, map visibility and URL parameters. A late response from an inactive panel must not replace the active mode's URL or steal focus. Reload recovery recomputes the requested route rather than treating a stored highlight as fresh computation. Directory place actions return to the normal campus planner.

Usability is also functional correctness. Contextual errors belong beside the relevant controls, directions remain available with the map hidden, and keyboard users need visible focus and meaningful tab behavior. Narrow layouts require readable controls without horizontal page scrolling. Physical-phone labels may require map zoom; this is distinct from hiding or truncating text directions.

15 Administrative graph and import workflows

The administrative application supports campus-scoped roles and authenticated sessions. Editors can modify drafts, reviewers can inspect submitted content, operations users manage operational information according to permissions, and administrators manage access. Session expiry, revocation, CSRF validation, password hashing and audit records support these workflows; they do not change shortest-path mathematics.

Staged data import

CSV, JSON and GeoJSON data enter a private staged workflow. Durable jobs process uploads, field mappings and row diagnostics before a preview is applied. Draft-version checks prevent a stale preview from overwriting later edits. Application is transactional, so an error partway through a batch must not leave a half-applied graph. Worker leases and bounded retries support restart recovery.

Graph editing

Outdoor and indoor editing support explicit nodes, connections, entrances and topology diagnostics. Split and snap operations must preserve deliberate connectivity. Deletion previews identify dependencies. Undo, redo and retained input improve editing, while server version checks remain the authority when multiple sessions race. An editor drawing two crossing lines has not necessarily authored a usable shared junction.

Review and publication

Validation checks structural consistency and produces warnings where evidence is incomplete. Review considers exact submitted content and warning reasons; publication exposes an immutable release. Separate application accounts are not the same as independent human or institutional approval when the same owner operates both. This project's acceptance evidence preserves that distinction.

None of these write workflows is required to activate the separate educational indoor graph. Keeping it read-only and outside campus releases prevents a classroom feature from silently changing the owner's already published MUJ/E22 data.

16 QR, location, closures and eligibility

The route engine operates in context. A QR marker can identify an installed start location, but the user still confirms being at that marker. A revoked or planned marker cannot be treated as an active route start. Reassigning a marker keeps its stable link while requiring fresh installation verification. These controls prevent an old sign from silently claiming a different valid location.

User-triggered location

Use my location requests a fresh browser fix only after a click. Candidate starts must be checked against eligible published paths, and landmark confirmation remains part of the implemented pilot flow. Low accuracy, unsupported position or expiry leads to manual or QR fallback. GPS does not determine an indoor floor and is not continuous live tracking in this submission.

Operational restrictions

Closures and lift outages change which edges are eligible. Estimated end times do not automatically certify that a path has reopened. A route computed before an overlay change must not remain displayed as though it were freshly validated. The public application checks restriction freshness and requires recalculation as appropriate.

Condition	Routing implication
Closed connection	Exclude affected movement
Stairs in step-free query	Exclude it
Unknown accessibility	Do not infer verified step-free status
Unknown physical distance	Exclude from metric objective where required
Directions-only request	Use one cost unit per eligible traversal
Educational demo	No real GPS, QR or closure connection is claimed

Filtering before search is important: finding a cheap invalid route and merely warning afterward would not solve the requested constrained path problem. The output must be computed on the graph that actually satisfies the selected policy.

17 Verification and observed results

Verification covers algorithm behavior, API contracts, rendering and retained-data preservation. Unit tests check specified graph cases, but passing examples alone are not a proof for all inputs and cannot show that a real door is open. Browser tests can exercise focus and reload behavior, but passing them alone cannot establish visual parity. SCNS therefore records test type, fixtures and limitations separately [5].

Evidence for integrated indoor delivery	Recorded result
Backend units reused from unchanged demo service	276 passed
PostGIS and API checks reused where unchanged	33 passed
Final frontend suite	76 passed; lint, types and build passed
Source browser coverage	91 distinct cases passed across broad run and affected rerun; one existing skip
Built nginx API worker browser suite	75 passed; one existing skip
Retention checks	22-table counts/content digests and private-storage digests matched

Representative assertions

- A → Library returns 3 traversals; A → C and its reverse return 9 with the correct stair direction.
- The isolated room produces no route; changing connections affects recomputed output.
- Physical distance and ETA remain null, and stale demo versions or foreign IDs are rejected.
- Outdoor and indoor responses, errors and URL state remain isolated across rapid mode changes.
- Existing directory, publication, QR, location and closure journeys retain their behavioral assertions.

The source result is not described as one uninterrupted all-green run: the broad run included two new locator failures, followed by a corrected affected rerun. Earlier failures and the retained transient button-transition contrast limitation are documented. Final Windows integrated-mode and physical-device checks remain owner-run rather than inferred from cloud fixtures.

18 Operation, limitations and future work

The demonstrated runtime uses Docker Desktop with Linux containers. The existing Compose stack separates a PostGIS database, FastAPI service, nginx-served frontend and import worker. Code updates must retain database volumes, private storage and environment configuration. The submission uses the established container runtime; native Windows deployment is outside its verified scope.

Presentation procedure

Start Docker Desktop and the retained database first, confirm database readiness, then start the API and frontend. Check the health-ready endpoint through port 5173 before opening the site. Show MUJ's directory and a known outdoor route, compare metric and traversal objectives, then switch to Indoor demo for same-floor, cross-floor and disconnected-room cases. Keep a backup recording and verify restart behavior before submission.

Current limits

MUJ outdoor connectivity and naming are incomplete. Real indoor MUJ rooms, surveyed entrance transitions, comprehensive step-free evidence and field-verified access rules are not available. The educational building is fictional and disconnected from MUJ outdoor navigation. GPS evidence in automated tests is simulated. Cloud tests do not certify performance, security or availability for a public production deployment.

Prioritized extension

Future work should first obtain authorized plans and dated observations, resolve real entrance anchors and tunnel topology, and add reviewed indoor room graphs. Further work can add issue reporting, deployment hardening, broader device/browser evaluation and a reproducible presentation dataset. Continuous live positioning requires a separate design for consent, accuracy, stale fixes, battery use and indoor uncertainty.

Algorithm extensions should follow a concrete need. A* could reduce explored outdoor nodes when a valid heuristic is available. BFS would be simpler for a dedicated equal-cost traversal mode. Neither should be claimed as implemented merely because it is a plausible improvement. The present contribution is a coherent, testable graph-based system with explicit data and evidence boundaries.

19 Conclusion and references

SCNS demonstrates an end-to-end application of DSA: model permitted movement as a directed graph, represent it with adjacency lists, use a heap-based Dijkstra search, track improvements with hash maps and reconstruct an ordered path from predecessors. Its interface turns that path into understandable map segments and directions. The two-floor educational graph makes topology, stair transitions and disconnected vertices visible during a classroom demonstration.

The main technical lesson is that route correctness has two parts. The algorithm must optimize the chosen costs correctly, and the underlying graph must represent valid movement. SCNS can compute an optimal result for eligible stored data while still honestly reporting that a building lacks an anchor or a physical measurement is unknown. This distinction is central to both the project's academic explanation and its future development.

Implementation references

[1] Pure routing engine. `backend/app/routing/dijkstra.py`

[2] Mixed graph cost and eligibility service. `backend/app/routing/mixed.py`

[3] Educational indoor graph and route service. `backend/app/services/educational_indoor.py`

[4] Indoor demo operation and graph semantics. `docs/operations/educational-indoor-demo.md`

[5] Verification and preservation evidence. `docs/testing/educational-indoor-demo.md`

[6] Catalog normalization and release lookup. `backend/app/services/catalog.py`

Data and algorithm references

[7] SCNS. MUJ v2 component inventory and recorded route results. Revision `136667f`.

[8] SCNS. Numbered MUJ naming atlas and source-supported E22 proposal. Revision `136667f`.

[9] Python Software Foundation. *heapq—Heap queue algorithm*, Python 3.12 documentation. `https://docs.python.org/3.12/library/heapq.html`.

[10] E. W. Dijkstra. "A note on two problems in connexion with graphs." *Numerische Mathematik* 1, 269–271 (1959). doi:10.1007/BF01386390.

[11] OpenStreetMap Foundation. *Copyright and License*. `https://www.openstreetmap.org/copyright`.

Repository reference and report facts rechecked on 19 October 2026 (India time). Screenshots are committed repository test renders, with metadata simulations disclosed in their captions. The complete application regression suite was not rerun for this report. A focused recheck of the downloaded routing code confirmed the worked example, all 49 indoor room pairs against an independent BFS, and distinct metric/traversal objectives. This does not verify the owner's current Windows database or physical routes.